SQL PORTFOLIO PROJECT

Finance Standard Cost 24-Month Rolling Forecast

Oracle Fusion | Cost Accounting | Forecasting | LAG() | UOM Conversion

Project Summary

This query builds a month-by-month rolling cost forecast for all active inventory items spanning up to 24 months. It joins standard or average cost methods, UOM conversion rates (to LB and CASE), and inventory movement flows. The result feeds finance's monthly cost-roll and variance pre-close analysis.

Thomas Foster

Data Analyst | Oracle Fusion ERP | SQL

github.com/thomasfoster | thomasfoster79@yahoo.com

1 PROJECT OVERVIEW

Platform:	Oracle Fusion ERP -- Inventory, Costing, Item Master
Query Type:	Rolling calendar CTE + LAG window function + cost method toggle
Grain:	Business Unit / Item / Month
Key Outputs:	Cost/unit, cost/LB, beginning/ending inventory LBs, transfer-in, shipment LBs
Parameters:	:p_start_month, :p_months (1-24), :p_cost_method (STANDARD|AVERAGE)
Audience:	Finance/FP&A, Cost Accounting, Supply Chain Planning

2 BUSINESS PROBLEM

Finance's cost-roll process required a forward-looking view of inventory value across up to 24 months -- but Oracle Fusion's standard reports only surfaced current-period costs. Analysts needed a parameterized query they could point at any starting month, dial up the horizon (1 = spot check, 24 = two-year forecast), and toggle between Standard and Average cost methods without changing the SQL.

- Monthly beginning and ending balances in LBs required a rolling LAG() calculation across the month series.
- UOM heterogeneity (CS, EA, LB) required on-the-fly conversion via EGP_UOM_CONVERSIONS.
- Gaps in transaction history had to produce zero-movement rows, not missing rows -- requiring CROSS JOIN of items x months.
- The query had to support both Standard Cost (CST_STD_COSTS) and Average Cost (CST_ITEM_COSTS) via a single parameter.
- Previous-month and next-month cost columns were needed side-by-side for trend comparison.

3 TECHNICAL EXPLANATION

- cal / months CTEs -- generate the month series using CONNECT BY LEVEL with ADD_MONTHS(), driven by :p_start_month and :p_months.
- org_bu -- maps inventory organizations to business units via FUN_ALL_BUSINESS_UNITS_V.
- item_dim -- joins EGP_SYSTEM_ITEMS_B to TL for item number, description, and primary UOM.
- to_lb / to_cs -- materialize UOM conversion rates using /*+ MATERIALIZE */ for performance.
- std_cost / avg_cost -- pull current standard or average cost; :p_cost_method governs which is used.
- txn / txn_mth -- classify inventory transactions into TRANSFER_IN, SHIPMENT, or OTHER and aggregate by item/org/month in LBs.
- item_org_months -- CROSS JOIN of all item/org combos x all months for continuity in zero-movement periods.
- flow_bal / flow_bal2 -- compute beginning inventory LBs using LAG() over the partition; derive end-of-month on-hand.
- Final SELECT -- surfaces prev/current/next month costs and quantities using separate LEFT JOINs with ADD_MONTHS() offsets.

Key Techniques: CONNECT BY calendar, CROSS JOIN, LAG(), MATERIALIZE hint, parameterized cost method, UOM conversion

4 MASKED SQL -- KEY EXCERPTS

```
-- Month calendar generator using CONNECT BY
cal AS (
  SELECT TRUNC(NVL(:p_start_month, SYSDATE),'MM') AS start_month,
         NVL(:p_months, 1)                        AS months_span
  FROM dual
),
months AS (
  SELECT ADD_MONTHS(c.start_month, LEVEL-1) AS month_start,
         ADD_MONTHS(c.start_month, LEVEL  ) AS month_end_excl
  FROM cal c CONNECT BY LEVEL <= c.months_span
),
-- UOM conversion materialised for performance
to_lb AS (
  SELECT /*+ MATERIALIZE */
         c.from_uom_code, c.conversion_rate AS to_lb_rate
  FROM egp_uom_conversions c WHERE c.to_uom_code = 'LB'
),
-- CROSS JOIN ensures every item x month has a row (no gaps)
item_org_months AS (
  SELECT i.organization_id, i.inventory_item_id, m.month_start
  FROM item_dim i CROSS JOIN months m
),
-- LAG() to compute rolling beginning balance
flow_bal AS (
  SELECT f.organization_id, f.inventory_item_id, f.month_start,
         f.transfer_in_lbs, f.shipment_lbs,
         NVL(LAG((NVL(f.transfer_in_lbs,0) - NVL(f.shipment_lbs,0)),1)
               OVER (PARTITION BY f.organization_id, f.inventory_item_id
                     ORDER BY f.month_start), 0) AS begin_lbs
  FROM flow f
),
-- Cost method toggle in final SELECT
SELECT
  bu.business_unit_name,
  i.item_number, i.description,
  TO_CHAR(m.month_start,'YYYY-MM')                          AS month,
  CASE WHEN UPPER(NVL(:p_cost_method,'STANDARD'))='STANDARD'
       THEN cc.unit_cost_std ELSE NULL END                  AS cost_per_unit,
  CASE WHEN UPPER(NVL(:p_cost_method,'STANDARD'))='STANDARD'
       THEN cc.unit_cost_std / NULLIF(lb_curr.to_lb_rate,0)
       ELSE NULL END                                        AS cost_per_lb,
  fb_curr.begin_lbs, fb_curr.transfer_in_lbs,
  fb_curr.shipment_lbs, fb_curr.onhand_lbs
FROM months m
JOIN flow_bal2 fb_curr ON fb_curr.month_start = m.month_start
JOIN item_dim i ON i.inventory_item_id = fb_curr.inventory_item_id
JOIN org_bu bu  ON bu.organization_id  = i.organization_id
LEFT JOIN cost_choice cc ON cc.inventory_item_id = i.inventory_item_id
LEFT JOIN to_lb lb_curr  ON lb_curr.from_uom_code = i.primary_uom_code
```

5 EXAMPLE OUTPUT

BU	Item #	Month	Cost/Unit	Cost/LB	Beg Inv LB	Xfer-In LB	Ship LB	On-Hand LB
[BU_A]	ITEM-020	2025-01	$3.10	$0.206	12,400	8,200	9,100	11,500
[BU_A]	ITEM-020	2025-02	$3.10	$0.206	11,500	7,800	8,600	10,700
[BU_A]	ITEM-021	2025-01	$1.85	$0.154	5,200	4,100	4,900	4,400
[BU_B]	ITEM-022	2025-01	$4.40	$0.275	2,100	1,800	2,000	1,900

** Business unit names, item numbers, and cost figures masked. Values illustrative.*